ROAD CONNECTION SYSTEM

OVERVIEW

The road connection system is what lights up roads and buildings that are connected. It determines if buildings are activated and able to create money/carbon. The primary script involved in this system is RoadAndResidenceConnectionManager.cs. It also uses the script ActivatableTile.cs, from which all activatable tiles inherit. (Which as a side note, in terms of coding practices, is not the best use of inheritance and caused me some struggles later on).

The way the system works is that every time a tile with the ActivatableTile class attached is placed or destroyed, the ActivatableTile class calls "RoadAndResidenceConnectionManager.current.UpdateResidenceConnections(this.gameObject)". This causes the connection manager to check and see if the tile it was provided is connected by roads to a system of roads where two buildings are connected. If so, it activates all the roads and buildings connected to the tile. If not, it deactivates all the roads and buildings connected to the tile. (Please note that it doesn't actually directly activate them, but rather it tells them that they are connected by roads, and then the tile can determine for itself if it should be activated. This is what allows factories to deactivate themselves if they don't have enough employees, even if they are connected by roads.)

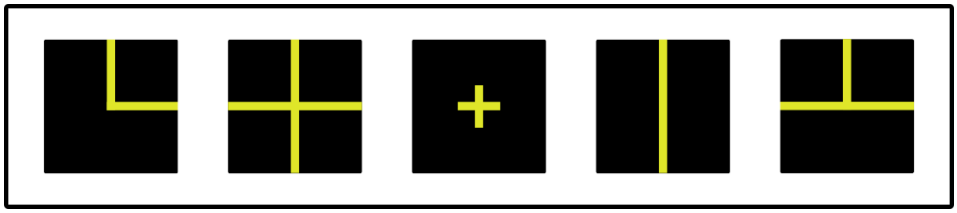
The way the connection manager searches for all connected tiles is with a simple flood search. A recursive function starts at the input tile and then adds all of its neighbors to a searched tiles list. It continues looking at all of those tiles' neighbors, ignoring tiles already in its list, until there are no tiles left to search. If there are two buildings in its searched tiles list, that means two buildings have been connected and so activates all of the tiles on its list at the end of the search. Otherwise, it deactivates all of the tiles on its list.

SIDE NOTE: VISUALLY CONNECTING ROADS

The way that roads visually connect to each other is a separate system from the one that activates connected roads/buildings, but this seemed like as good a place as any to explain how they visually connect.

The way that roads visually connect is handled by the script RoadConnections.cs, which is put on road tiles. The RoadConnections class's function UpdateModelConnections(bool updateNeighborConnections) updates the road's material so that its lines visually connect to the surrounding roads. A road's visual connections are updated whenever it's placed and also every time it moves when begin dragged around by the player during the placement process.

The road always has 5 materials to choose from. These are what they look like:



By rotating these five materials, it can show connections to any combination of its neighbors, or by using the middle one, it can show that it's connected to none of them. The way it determines which material to use and which way to rotate it is admittedly slightly janky. I basically just wrote out all 16 possible different connections of neighbors it could have, and for each of those cases wrote out which material to use and which way to rotate it. That information is stored in these three arrays in the RoadConnections class: neighborAlignments (all 16 possible combinations of neighbors), arrangementOrientationAngles (which way to rotate the model for each combination of neighbors), and arrangementModels (which model to use for each combination). The way the arrangements are stored is definitely not as efficient as it could be (I stored each arrangement as an integer, like this: 1101 or 1011). Feel free to fix that if you have the time.

INTERNAL API

These are some of the important public functions of the RoadAndResidenceConnectionManager class.

bool AllResidencesAreConnected()

Returns true if all the residences on the active chunk are connected. Returns false otherwise. Note: this function ignores activatable tiles that aren't residences.

void UpdateResidenceConnections(GameObject objectToCheck)

Updates the connection status for all the tiles around the input tile, "objectToCheck." It disables all the connected tiles if two buildings aren't connected in that connection system; otherwise, if two buildings are connected, it activates all the connected tiles.

GameObject[] GetRoadNeighbors(Vector3 tileLocation)

Returns an array of all the activatable tiles neighboring the tile at the input tileLocation.

-Graydon Bruyn, May 2026